

MVC (Model View Controller)

Data Access Object(DAO) :-

What is DAO ?

- The Data Access Object (DAO) Design Pattern separates database logic from business logic.
- It provides a dedicated layer for handling all data operations like insert, update, delete, and fetch.
- DAO hides the internal details of how data is stored and retrieved.
- It offers a simple interface so the rest of the application can access data without dealing with SQL or JDBC code.
- This pattern improves maintainability, reusability, and reduces coupling between application layers.

Why Do We Need the Data Access Object (DAO) ?

1. Separation of Concerns

- DAO keeps all database operations (CRUD) in one layer.
- Business logic doesn't need to know SQL or database details.

2. Easy Maintenance

- If the database changes (e.g., MySQL → PostgreSQL), only the DAO layer needs updating.

3. Reusability

- DAO methods can be reused anywhere (e.g., `saveUser()`, `getAllProducts()`).

4. Security & Clean Code

- Prevents SQL being scattered across the project.
- Reduces duplication and errors.

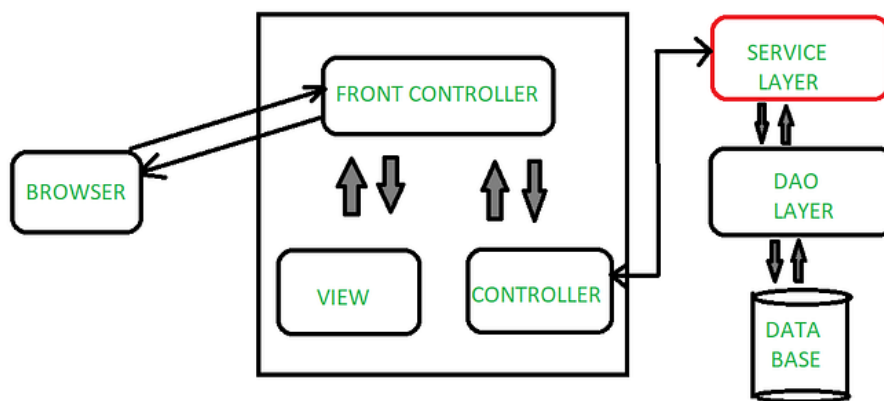
5. Easy Testing (Mocking)

- The DAO layer can be mocked for unit tests.
- Helps test business logic without connecting to a real database.

6. Loose Coupling

- The application depends on interfaces, not direct database code.

Why to use Service Layer in Spring MVC ?



1. To separate business logic from controllers.

- Controllers should only handle requests and responses—not calculations, validations, or processing.
- The Service Layer performs that work.

2. To make the code clean and organized.

- Instead of writing complex logic in controllers or DAO, the service layer holds all business rules.

3. To avoid mixing database code with business logic.

- DAO does only database operations.
- Services decide *how* data should be processed before/after DB calls.

4. To improve reusability.

- Multiple controllers can use the same service method.

5. To make applications easier to test.

- Business logic in service classes can be tested without touching controllers or database.

6. To support scalable architecture.

- When your project grows, service layer helps add more features without breaking controllers/DAO.

7. To follow proper layered architecture (MVC + Service + DAO).

- This promotes loose coupling and single responsibility principle.

Common DAO Operations :-

Operation	Method Name	Purpose
Create	save() / insert()	Insert a new record into the database
Read (Single)	findById()	Fetch one record using primary key
Read (All)	findAll()	Fetch all records from the table
Update	update()	Modify an existing record
Delete	deleteById()	Remove a record using primary key
Exists Check	existsById()	Check if a record exists
Count Records	count()	Get total number of records
Custom Query	findByField()	Fetch records using custom conditions
Search / Filter	search()	Apply filters, keywords, or conditions
Batch Insert	saveAll()	Insert multiple records at once
Batch Delete	deleteAll()	Delete multiple records at once

Advantages of DAO :-

1. Abstraction & Encapsulation

- Hides database details and provides a clean interface.

2. Centralized Data Logic

- All DB operations are organized in one place, making code easy to maintain.

3. Flexibility

- Easy to switch databases or change queries without affecting other layers.

4. Better Testability

- Allows mock DAOs for testing without a real database.

5. Security & Validation

- Central place to validate inputs and prevent SQL injection.

Disadvantages of DAO :-

1. Extra Complexity

- Adds unnecessary layers for small applications.

2. Code Duplication

- Similar CRUD code may repeat in multiple DAOs.

3. Learning Curve

- Beginners may find DAO structure difficult at first.

4. Overhead for Small Projects

- More classes and interfaces make small apps bulky.

5. Minor Performance Cost

- Extra abstraction layers can cause slight performance overhead.

Steps to Create a Spring MVC Student Form :-

To configure the Spring MVC web app, start by adding the required dependencies in the pom.xml. This file tells Maven which libraries to download and manage for your application.

```
http://maven.apache.org/maven-v4_0_0.xsd (xsi:schemaLocation with catalog)
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.demos</groupId>
  <artifactId>Maven_Sample_Project2</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>Maven_Sample_Project2 Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.32</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-jdbc</artifactId>
      <version>5.3.32</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.postgresql/postgresql -->
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.7.8</version>
    </dependency>
  </dependencies>
  <build>
    <finalName>Maven_Sample_Project2</finalName>
  </build>
</project>
```

The web.xml file maps URLs to servlets that handle incoming requests. It includes a startup listener, servlet configuration, and servlet-mapping. The servlet is named spring, which will be used later in the application.

Bind to grammar/schema...

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">

    <servlet>
        <servlet-name>spring</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>spring</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>

</web-app>
```

Create a Student.java model class in the com.demo.model package with fields like id, fullname, email, password & address. These fields will bind to the input fields on the view page.

```
package com.demo.models;

public class Student {
    private int id;
    private String fullname;
    private String email;
    private String password;
    private String address;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getFullname() {
        return fullname;
    }
    public void setFullname(String fullname) {
        this.fullname = fullname;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}
```

The index.jsp defines the student login page components with the appropriate fields to match the student object, this file is located in the "/src/main/webapp/WEB-INF/pages/index.jsp".

```
<html>
<body>

    <!-- <form action="add" method="post">
        <input type="text" name="num1" placeholder="Enter number1"><br><br>
        <input type="text" name="num2" placeholder="Enter number2"><br><br>

        <button type="submit">Submit</button>
    </form> -->

    <form action="save" method="post">
        <input type="text" name="fullname" placeholder="Enter fullname"><br><br>
        <input type="email" name="email" placeholder="Enter email"><br><br>
        <input type="password" name="password" placeholder="Enter password"><br><br>
        <textarea name="address" id="address" placeholder="Enter address"></textarea><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```


The spring-servlet.xml file (in *WEB-INF*) handles all HTTP requests for the application, enables @Controller, and manages resource handling. It also configures view resolution and uses component-scan to detect annotated beans.

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation) | http://www.
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:mvc="http://www.springframework.org/schema/mvc"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd
    http://www.springframework.org/schema/mvc
    http://www.springframework.org/schema/mvc/spring-mvc.xsd">

  <context:component-scan base-package="com.demo.controllers"></context:component-scan>

  <!-- View Resolver -->
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/" />
    <property name="suffix" value=".jsp" />
  </bean>

  <bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="org.postgresql.Driver" />
    <property name="url" value="jdbc:postgresql://localhost:5433/springmaven1" />
    <property name="username" value="postgres" />
    <property name="password" value="4518" />
  </bean>

  <bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource" />
  </bean>

  <bean id="dao" class="com.demo.dao.StudentDAO">
    <property name="template" ref="jdbcTemplate"></property>
  </bean>

  <!-- Enable Spring MVC -->
  <mvc:annotation-driven />
</beans>
```

The controller class processes client requests, executes business logic, and returns the appropriate view. In StudentController, the @GetMapping loads the login page with model data, while the @PostMapping validates the user input and redirects to studentsdata.jsp.

```
package com.demo.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.demo.dao.StudentDAO;
import com.demo.models.Student;

@Controller
public class StudentController {

    // @PostMapping("/add")
    // public String addition(@RequestParam int num1, @RequestParam int num2, Model m)
    // {
    //     int result=num1+num2;
    //     System.out.println("Addition= "+result);
    //     m.addAttribute("result",result);
    //     return "welcome";    //welcome.jsp
    // }

    // @GetMapping("/welcome")
    // public String wel() {
    //     return "welcome";    //welcome.jsp
    // }

    @Autowired
    StudentDAO sado;

    @PostMapping("/save")
    public String save(@ModelAttribute Student s) {
        boolean result=sado.registration(s);
        if(result) {
            System.out.println("Registration Successfully Completed");
            return "redirect:/studentsdata";
        }else {
            System.out.println("Fail");
            return "redirect:/";
        }
    }

    @GetMapping("/studentsdata")
    public String studentsdata() {
        return "studentsdata";    //studentsdata.jsp
    }
}
```

After a successful login, StudentController redirects to studentsdata.jsp (in *WEB-INF/pages*), which contains a simple welcome HTML page.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Students Data</h1>
</body>
</html>
```

The StudentDAO class uses JdbcTemplate to handle database operations. Its registration() method inserts a new Student record into the database using a parameterized SQL query.

```
package com.demo.dao;

import org.springframework.jdbc.core.JdbcTemplate;
import com.demo.models.Student;

public class StudentDAO {

    JdbcTemplate template;

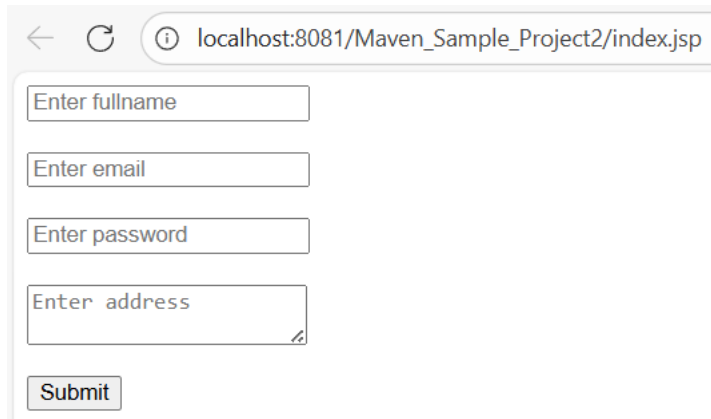
    public void setTemplate(JdbcTemplate template) {
        this.template = template;
    }

    public boolean registration(Student s) {
        // String sql="insert into student(fullname,email,password)
        // values('"+s.getFullname()+"','"+s.getEmail()+"','"+s.getPassword()+"','"+s.getAddress()+"')";
        // template.update(sql);

        String sql="insert into student(full_name,email,password,address) values(?,?,?,?)";
        template.update(sql,s.getFullname(),s.getEmail(),s.getPassword(),s.getAddress());
        return true;
    }
}
```

Output :-

Run the project on Tomcat and open: http://localhost:8081/Maven_Sample_Project2/index.jsp



localhost:8081/Maven_Sample_Project2/index.jsp

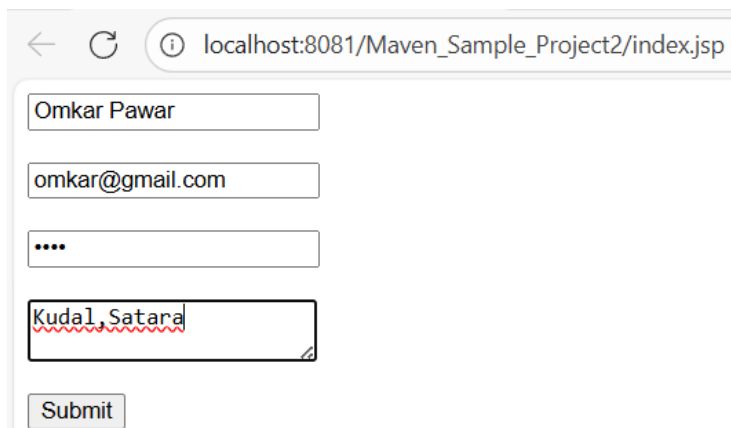
Enter fullname

Enter email

Enter password

Enter address

Submit



localhost:8081/Maven_Sample_Project2/index.jsp

Omkar Pawar

omkar@gmail.com

....

Kudal, Satara

Submit



localhost:8081/Maven_Sample_Project2/studentsdata

Students Data

Console Based -

Registration Successfully Completed

Database Based -

	id [PK] integer	full_name character varying (100)	email character varying (100)	password character varying (100)	address character varying (255)
1	1	Sagar Kate	sagarkate374@gmail.c...	2580	[null]
2	4	Omkar Pawar	omkar@gmail.com	4580	Kudal,Satara